

Tendências de Motores de Jogos

Semana 10

Inventário e Ampliação do Interaction System

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 3 — Resolver Problemas

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender padrões universais de Inventory System (armazenamento, adição/remoção, exibição)
- Estruturar um `InventoryComponent` funcional, reutilizando os itens já modelados em `DT_Items` (Semana 6)
- Retomar e ampliar o `InteractionComponent` / `BPI_Interactable` (Semana 5) para múltiplos tipos de interação
- Conectar o Interaction System ao Inventário (coletar, usar, descartar), com um novo tipo proposto por cada grupo

Como a semana está organizada



Encontro 1

Inventory System — `InventoryComponent`, reutilizando `DT_Items` da Semana 6



Encontro 2

Ampliação do Interaction System — múltiplos tipos de interação conectados ao inventário +
Code Review

Tendências de Motores de Jogos

ENCONTRO 1

Inventory System e InventoryComponent

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Os itens do seu jogo já existem. Mas alguém sabe quais o jogador possui?

Pense na diferença entre o dado de um item existir e o estado de posse desse item ser controlado por algum sistema.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Separar o dado do item da posse do item

- `DT_Items` já define o que um item é — nome, tipo, atributos (Semana 6)
- Falta um sistema que controle quais itens, e quantos, o jogador possui agora
- Um Inventory System não redefine o item — apenas gerencia uma coleção de referências a itens já definidos
- O mesmo padrão de Component isolado já usado por `HealthComponent` e `InteractionComponent`

DICA

Se o dano de um item mudar em `DT_Items`, nenhum outro sistema deveria precisar ser atualizado manualmente.

InventoryComponent : armazenamento, adição/remoção, exibição

- Component dedicado, anexado ao `BP_Player`, seguindo o padrão de `HealthComponent` / `InteractionComponent`
- Mantém uma coleção interna (Array ou Map) de referências a linhas de `DT_Items`
- Expõe funções de adicionar, remover e consultar itens
- O restante do projeto nunca precisa conhecer a estrutura interna dessa coleção

Inventário: Unreal × Unity

Unreal

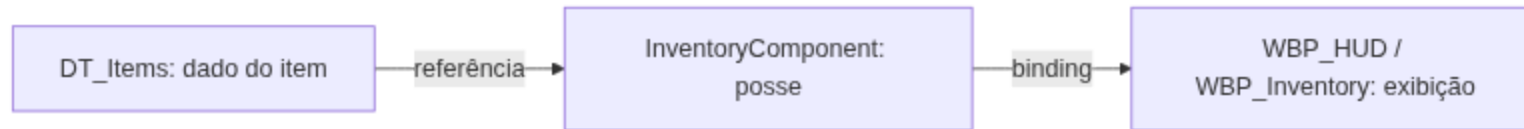
`InventoryComponent` (Component)
mantendo Array/Map de referências a
linhas de `DT_Items` (Data Table + Struct)

Unity

`MonoBehaviour` de inventário mantendo
`List<T>` / `Dictionary` de referências a
`ScriptableObject`s de item

Tendências de Motores de Jogos

Item, posse e exibição



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Demonstração: `InventoryComponent` com adicionar item

O professor cria um `InventoryComponent`, com Array interno de itens possuídos e uma função de adicionar item por referência a uma linha de `DT_Items`.

Resultado esperado: ao chamar a função, o item passa a constar na coleção interna do Component, consultável por outros sistemas.

Imagem sugerida

*Objetivo: mostrar o Event Graph de um **InventoryComponent** com a função de adicionar item.*

Enquadramento: captura de tela do editor de Blueprint da Unreal, Event Graph em primeiro plano.

*Elementos presentes: nó de função "Add Item", referência a uma linha de **DT_Items**, Array de itens possuídos exposto no painel de variáveis.*

Destaque visual: circular o nó que adiciona a referência ao Array interno.

Legenda sugerida: "InventoryComponent adicionando uma referência de DT_Items ao Array de itens possuídos."

Boas práticas

✓ BOAS PRÁTICAS

- Nomear o Component com o prefixo `BP_` seguido de `Component` (`InventoryComponent`), conforme `PROJECT_ARCHITECTURE.md`
- Armazenar apenas referências às linhas de `DT_Items` , nunca duplicar seus atributos
- Manter o `InventoryComponent` desacoplado de qualquer lógica de UI

Laboratório do Encontro 1

Cada grupo estrutura seu próprio `InventoryComponent`, reutilizando os itens já modelados em `DT_Items` na Semana 6, e implementa adição e remoção básica.

OBJETIVOS

Critério de sucesso: o `InventoryComponent` armazena e consulta itens referenciados em `DT_Items`, sem alterar a lógica de nenhum sistema anterior.

Fechando o Encontro 1

- Inventory System separa o dado do item (`DT_Items`) da posse do item (`InventoryComponent`)
- Cada grupo com um `InventoryComponent` funcional, armazenando e consultando itens
- Próximo encontro: conectar esse inventário ao Interaction System da Semana 5

Tendências de Motores de Jogos

ENCONTRO 2

Ampliando o Interaction System

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Uma única reação de interação resolve coletar, usar e descartar itens?

Pense no que muda quando um sistema de interação precisa distinguir tipos diferentes de resposta, não apenas disparar uma reação genérica.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

De uma reação genérica a múltiplas respostas

- Semana 5: `InteractionComponent` detectava e disparava uma única chamada via `BPI_Interactable`
- Agora: a interação precisa distinguir coletar, usar e descartar, conectados ao `InventoryComponent`
- O princípio de desacoplamento não muda — muda a granularidade da resposta
- `BPI_Interactable` continua sendo o único ponto de comunicação entre `InteractionComponent` e o mundo



DICA

O `InteractionComponent` nunca deveria precisar conhecer se o objeto interativo é um item, uma porta ou uma alavanca.

Recursos retomados e novos

- `InteractionComponent`, `BPI_Interactable`, Event Dispatchers — retomados da Semana 5
- `InventoryComponent` — construído no Encontro 1
- Conexão: coletar (adicionar ao inventário), usar (aplicar efeito), descartar (remover e devolver ao mundo)

Múltiplos tipos de interação: Unreal × Unity

Unreal

`BPI_Interactable` com parâmetro de tipo, ou interfaces adicionais especializadas por tipo de interação

Unity

Interface C# equivalente, distinguindo tipos por parâmetro (enum) do método ou por múltiplas interfaces especializadas

Fluxo de uma interação de coleta



Demonstração: `BP_Pickup` conectado ao inventário

O professor conecta um Actor interativo (`BP_Pickup`) que implementa `BPI_Interactable` e, ao ser interagido, adiciona seu item ao `InventoryComponent` do jogador.

Resultado esperado: ao interagir com o `BP_Pickup`, o item correspondente passa a constar no inventário do jogador.

Imagem sugerida

*Objetivo: mostrar a interface **BPI_Interactable** implementada por um **BP_Pickup** que se comunica com o **InventoryComponent**.*

Enquadramento: captura de tela do editor de Blueprint, com dois grafos lado a lado.

*Elementos presentes: grafo do **BP_Pickup** chamando a interface; grafo do **InteractionComponent** recebendo o evento e acionando o **InventoryComponent**.*

Destaque visual: seta tracejada conectando os dois grafos através da interface, sem linha direta entre as classes.

Legenda sugerida: "BP_Pickup e InteractionComponent comunicando-se por BPI_Interactable, sem referência direta."

Desafio: ampliando o Interaction System

Cada grupo expande seu sistema de interação para suportar um novo tipo além de coletar — empilhar itens, combinar itens, ou interação com cooldown — conectado ao `InventoryComponent`, com solução própria.

ATENÇÃO

Não há indicação prévia de qual novo tipo implementar. A escolha técnica faz parte do desafio.

Boas práticas

✓ BOAS PRÁTICAS

- Nunca contornar `BPI_Interactive` com `Cast to` direto entre o Actor interativo e o `InventoryComponent`
- Modularizar a lógica comum entre coletar, usar e descartar, evitando duplicação
- Manter a nomenclatura consistente com `InteractionComponent` / `InventoryComponent` já estabelecidos

Code Review — Inventário e Interação

Ao final do encontro, cada grupo apresenta os sistemas de inventário e interação ampliada para Code Review formal, terceira aplicação da Rubrica 4 no semestre.

OBJETIVOS

Critérios avaliados: Organização dos Blueprints, Nomenclatura, Modularidade, Reutilização, Comunicação entre sistemas, Boas práticas gerais.

Resumo da Semana 10

- Inventory System separa o dado do item (`DT_Items`) da posse do item (`InventoryComponent`)
- Interaction System evolui de uma reação genérica para múltiplas respostas (coletar, usar, descartar), via `BPI_Interactable`
- Terceiro desafio de liberdade real de solução da disciplina, avaliado por Code Review formal
- Nenhum sistema dos Módulos 1 e 2 substituído ou quebrado

Checklist da Semana

OBJETIVOS

- `InventoryComponent` funcional, armazenando e consultando itens de `DT_Items` (Encontro 1)
- `InteractionComponent` ampliado, suportando coletar e ao menos um segundo tipo de interação (Encontro 2)
- Conexão entre inventário e interação via `BPI_Interactable`, sem `Cast to` direto
- Code Review formal (Rubrica 4) realizado

Próximos passos



DICA

A Semana 11 introduz Navigation, Behavior Trees e Blackboards, encerrando a Unidade III com a entrega do Vertical Slice jogável do Módulo 3 — animação, interface, inventário, interação ampliada e IA integrados —, seguida de Playtest coletivo e Showcase.

Leitura recomendada: Gameplay Framework in Unreal Engine — Data Assets aplicados a inventário (Epic Games Documentation).